Supabase Security for Vibe Coders: Improve the Security of Your App

If you're building with Cursor, Bolt, Lovable, v0, or any AI coding tool and using Supabase as your backend, this guide is for you. I'm not a security expert, but I've built enough Supabase apps to run into most of the common issues. This is a practical guide covering schema design, Row Level Security (RLS), and the specific mistakes AI assistants make that leave your data exposed.

> **How to use this:** Drop this file into your project root. Your AI coding assistant (Claude, Cursor, Copilot) will pick it up as context and follow these practices when generating Supabase code. Instead of fixing security after the fact, your AI will get it right from the start.

Another hint is that you can use the **Supabase Advisor** to help you create the necessary changes in your Supabase 🙂

---

1. Schema Design: Minimize What You Store

Don't duplicate PII from `auth.users`

Supabase already stores emails, phone numbers, and metadata in `auth.users`. This table is **never exposed** through the PostgREST API — it's protected by default. When you copy that data into your own tables, you create a second copy that you now have to protect yourself.

```sql
-- BAD: Duplicating email into your public table
CREATE TABLE public.profiles (
  id uuid REFERENCES auth.users PRIMARY KEY,
  email text NOT NULL,        -- Duplicated from auth.users
  name text,
  role text
);
-- If RLS is misconfigured, attacker gets all emails instantly
```

```
-- GOOD: Only store what auth.users doesn't have
CREATE TABLE public.profiles (
  id uuid REFERENCES auth.users PRIMARY KEY,
  display_name text,
  role text,
  timezone text,
  avatar_url text
);
-- Even if RLS fails, no emails/passwords/PII leak
```

When you need the email (e.g., for display), fetch it from the auth context on the client side (`user.email`) or join via a server-side function using the service role key. Never expose it through a public table.

Think about what each column leaks

Before adding a column to a table, ask: "If this entire table leaked, what's the damage?"

Column	Risk if leaked	Recommendation
`email`	High — spam, phishing, GDPR	Keep in `auth.users` only
`name`	Medium — identity	Keep in profiles, protect with RLS
`role`	Medium — helps attackers target admins	Protect with RLS
`internal_notes`	High — private observations about users	Move to a separate table with strict RLS

`goal_text`	Low-Medium — personal but not secret	Protect with RLS
`id` (uuid)	Low — not useful alone	Fine in any table

Separate sensitive columns into their own tables

If a table has a mix of public and sensitive fields, and RLS can only filter rows (not columns), split it:

```sql
-- Instead of internal_notes on the users table (visible to
teammates):
ALTER TABLE public.users DROP COLUMN internal_notes;

CREATE TABLE public.admin_notes (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES public.users NOT NULL,
  author_id uuid REFERENCES auth.users NOT NULL,
  notes text,
  updated_at timestamptz DEFAULT now()
);

-- Now only the author and admins can see these
CREATE POLICY "Authors read own notes"
  ON public.admin_notes FOR SELECT
  USING (author_id = auth.uid());
```

---

2. Understanding the Supabase Security Model

The anon key is always public

Your `NEXT_PUBLIC_SUPABASE_ANON_KEY` ships in the browser JS bundle. This is by design — it's not a secret. The security model is:

- **Anon key** = "front door key" — gets you to the API, but RLS decides what you see
- **Service role key** = "master key" — bypasses RLS entirely, server-side only

```
None
NEXT_PUBLIC_SUPABASE_URL=...          # Public, fine
NEXT_PUBLIC_SUPABASE_ANON_KEY=...     # Public, fine (RLS is your
guard)
SUPABASE_SERVICE_ROLE_KEY=...         # PRIVATE — server-side
only, never NEXT_PUBLIC_
```

Middleware and UI are not security

Route guards, middleware redirects, and hidden UI elements are for user experience. An attacker bypasses your entire app with `curl`:

```
Shell
# This ignores your Next.js middleware, React router, and all UI
guards
curl "https://your-project.supabase.co/rest/v1/profiles?select=*"
\
  -H "apikey: your-public-anon-key"
```

RLS is the only thing between your data and the internet.

Schema introspection leaks table names

By default, Supabase exposes an OpenAPI spec at `/rest/v1/` that reveals all table names and RPC functions. An attacker uses this to map your database before targeting specific tables. Disable it in **Project Settings > API > Exposed schemas** if you don't need it publicly.

---

3. RLS Setup: The Complete Checklist

For every table, answer these questions

Question	If No...

Is RLS enabled?	`ALTER TABLE public.my_table ENABLE ROW LEVEL SECURITY;`
Does it have a SELECT policy?	Data is exposed to anyone with the anon key
Does the SELECT policy check `auth.uid()`?	Unauthenticated users can read everything
Does it have INSERT/UPDATE/DELETE policies?	Writes are open to anyone
Are policies scoped to the correct role?	Users may see/edit data they shouldn't

The #1 mistake: missing SELECT policies

When you enable RLS and add write policies but forget SELECT policies, reads stay open. The Supabase dashboard's "Enable read access for all users" quick toggle creates a policy with `USING (true)` — anyone with the anon key reads everything.

```sql
-- BAD: What the dashboard quick toggle creates
CREATE POLICY "Enable read access for all users"
  ON public.profiles FOR SELECT
  USING (true);  -- Wide open

-- GOOD: Require authentication at minimum
CREATE POLICY "Authenticated users can read"
  ON public.profiles FOR SELECT
  USING (auth.uid() IS NOT NULL);

-- BETTER: Only own data
CREATE POLICY "Users read own profile"
```

```
  ON public.profiles FOR SELECT
  USING (auth.uid() = id);
```

Audit queries to find gaps

```SQL
-- Find tables with RLS disabled
SELECT c.relname AS table_name, c.relrowsecurity AS rls_enabled
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'public' AND c.relkind = 'r'
ORDER BY c.relname;

-- Find tables missing SELECT policies
SELECT DISTINCT t.tablename
FROM pg_tables t
WHERE t.schemaname = 'public'
AND t.tablename NOT IN (
  SELECT tablename FROM pg_policies
  WHERE schemaname = 'public' AND cmd = 'SELECT'
);

-- View all policies
SELECT tablename, policyname, cmd, qual
FROM pg_policies
WHERE schemaname = 'public'
ORDER BY tablename, cmd, policyname;
```

---

4. Helper Functions for RLS

When a policy on `profiles` needs to read from `profiles` (e.g., checking the user's role), you get infinite recursion. Solve this with `SECURITY DEFINER` functions that bypass RLS internally:

```sql
CREATE OR REPLACE FUNCTION public.get_my_role()
RETURNS text
LANGUAGE sql
SECURITY DEFINER  -- Runs with elevated privileges, bypasses RLS
STABLE            -- Result can be cached within a transaction
AS $$
  SELECT role FROM public.profiles WHERE id = auth.uid()
$$;
```

Common helpers for multi-role apps

```sql
-- Check ownership of a scoped resource (e.g., "is this my
team?")
CREATE OR REPLACE FUNCTION public.is_member_of_team(p_team_id
uuid)
RETURNS boolean LANGUAGE sql SECURITY DEFINER STABLE AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.team_members
    WHERE user_id = auth.uid() AND team_id = p_team_id
  )
$$;

-- Current user's record ID in a child table
CREATE OR REPLACE FUNCTION public.get_my_member_id()
RETURNS uuid LANGUAGE sql SECURITY DEFINER STABLE AS $$
  SELECT id FROM public.members WHERE user_id = auth.uid()
$$;
```

---

5. Policy Design Patterns

Pattern A: Authenticated-only (shared/reference data)

For tables where all logged-in users need read access — lookup tables, shared schedules, configuration:

```sql
CREATE POLICY "Authenticated read"
  ON public.plans FOR SELECT
  USING (auth.uid() IS NOT NULL);
```

Pattern B: Own-record only (personal data)

For tables where users should only see their own rows:

```sql
-- Direct ownership via user_id column
CREATE POLICY "Users read own record"
  ON public.profiles FOR SELECT
  USING (auth.uid() = id);

-- Indirect ownership via a lookup function
CREATE POLICY "Members read own attendance"
  ON public.attendance FOR SELECT
  USING (member_id = public.get_my_member_id());
```

Pattern C: Role-based hierarchy (admin > manager > user)

For tables with tiered access, create one policy per role:

```sql
-- Admin sees everything
CREATE POLICY "Admins read all"
  ON public.members FOR SELECT
  USING (public.get_my_role() = 'admin');

-- Manager sees their team
CREATE POLICY "Managers read team members"
  ON public.members FOR SELECT
  USING (
    public.get_my_role() = 'manager'
    AND public.is_member_of_team(team_id)
```

```
  );

-- User sees own record
CREATE POLICY "Users read own record"
  ON public.members FOR SELECT
  USING (user_id = auth.uid());
```

Pattern D: Peer access (teammates, partners)

When users need to see specific other users' data:

```SQL
-- Users in the same team can see each other
CREATE POLICY "Teammates can read each other"
  ON public.members FOR SELECT
  USING (
    team_id IS NOT NULL
    AND team_id = public.get_my_team_id()
  );
```

Combining patterns

Multiple SELECT policies on the same table are OR'd together (any matching policy grants access). This lets you layer them:

```SQL
-- A user can read a member row if ANY of these are true:
-- 1. They're an admin
-- 2. They're the manager of that team
-- 3. It's their own record
-- 4. They're on the same team
```

---

6. Write Policies Are Just as Critical as SELECT

Most guides focus on SELECT policies, but missing or weak write policies are equally dangerous.

Privilege escalation via UPDATE

If your `profiles` table has a `role` column and the UPDATE policy is just `USING (id = auth.uid())`, any user can change their own role to `admin`:

```shell
# Any authenticated user promotes themselves to admin
curl -X PATCH
"https://your-project.supabase.co/rest/v1/profiles?id=eq.MY_ID" \
  -H "Authorization: Bearer MY_JWT" \
  -d '{"role": "admin"}'
# Returns: [{"role": "admin"}]  — user is now admin
```

Fix with `WITH CHECK` that locks the role field:

```sql
-- Users can update own profile but role must stay the same
CREATE POLICY "Users can update own profile"
  ON public.profiles FOR UPDATE
  USING (id = auth.uid())
  WITH CHECK (
    id = auth.uid()
    AND role = (SELECT role FROM public.profiles WHERE id =
auth.uid())
  );

-- Only admins can change roles
CREATE POLICY "Admins can update any profile"
  ON public.profiles FOR UPDATE
  USING (public.get_my_role() = 'admin');
```

Self-registration via INSERT

If an INSERT policy on your `users` or `members` table is too permissive, anyone can create records with elevated status:

```shell
# Attacker creates themselves as an active member, bypassing approval
curl -X POST "https://your-project.supabase.co/rest/v1/members" \
  -H "Authorization: Bearer MY_JWT" \
  -d '{"user_id": "MY_ID", "status": "active", "is_verified": true}'
```

Fix by constraining what self-created records can contain:

```sql
-- Self-registration only allowed with safe defaults
CREATE POLICY "Users can create own record"
  ON public.members FOR INSERT
  WITH CHECK (
    user_id = auth.uid()
    AND status = 'pending'       -- Can't self-activate
    AND team_id IS NULL           -- Can't self-assign to a team
  );

-- Admins can insert anything
CREATE POLICY "Admins can insert members"
  ON public.members FOR INSERT
  WITH CHECK (public.get_my_role() = 'admin');
```

Sensitive field writes (internal_notes, admin fields)

Even if a table has proper UPDATE policies, make sure users can't write to fields they shouldn't:

```sql
-- BAD: Any team member can update any field, including internal_notes
CREATE POLICY "Team members can update"
```

```
  ON public.members FOR UPDATE
  USING (team_id = public.get_my_team_id());

-- GOOD: Only managers and admins can update member records
CREATE POLICY "Managers can update their team members"
  ON public.members FOR UPDATE
  USING (
    public.get_my_role() = 'manager'
    AND public.is_member_of_team(team_id)
  );
```

Write policy checklist

For every table, verify:

- **INSERT**: Who can create rows? Can they set any field to any value?
- **UPDATE**: Can users change fields they shouldn't (role, status, admin-only fields)?
- **DELETE**: Who can delete? Can a user delete other users' data?

---

7. Auth Configuration

Enable email confirmation

By default, Supabase auto-confirms email signups — meaning a throwaway `attacker@mailinator.com` gets instant authenticated access. This is the difference between "anyone on the internet" and "anyone who controls a real email address."

Enable in: **Supabase Dashboard > Authentication > Confirm sign up > Enable email confirmations**

Without email confirmation, your "authenticated users can read" policies effectively mean "anyone can read."

Open signup means anyone becomes "authenticated"

If your app has a signup page, anyone can create an account. Combined with `auth.uid() IS NOT NULL` policies, this means any person on the internet is one free signup away from reading all your "authenticated-only" data.

Consider whether you actually need open signup, or if users should be invite-only (admin creates accounts via service role).

---

8. Common Gotchas

NULL comparisons silently break policies

In SQL, `NULL = NULL` is `FALSE`. If a column can be NULL, your policy might silently exclude valid rows:

```sql
-- BUG: If team_id is NULL for unassigned users, this never
matches
USING (team_id = public.get_my_team_id());

-- FIX: Guard against NULL
USING (
  team_id IS NOT NULL
  AND team_id = public.get_my_team_id()
);
```

Scope changes break history access

If a user moves from Team A to Team B, their old data still has `team_id = Team A`. A team-scoped policy alone hides their own history:

```sql
-- BUG: User loses access to their own old data after changing
teams
CREATE POLICY "Users read team data"
  ON public.tasks FOR SELECT
  USING (team_id = public.get_my_team_id());
```

```
-- FIX: Add a separate own-record policy
CREATE POLICY "Users read own tasks"
  ON public.tasks FOR SELECT
  USING (user_id = public.get_my_member_id());
```

Joins respect RLS per-table independently

When you do `.select('*, tasks:tasks(*)')`, RLS on the `tasks` table independently filters the joined rows. A user might see the parent row but get empty nested arrays. This is usually correct — but be aware the UI might show empty states.

CREATE OR REPLACE can't rename function parameters

If existing policies depend on a function, you can't change its parameter names without dropping everything first:

```SQL
-- ERROR: cannot change name of input parameter "p_team_id"
CREATE OR REPLACE FUNCTION is_member_of_team(team_id uuid) ...

-- FIX: Keep the original parameter name
CREATE OR REPLACE FUNCTION is_member_of_team(p_team_id uuid) ...

-- OR: DROP CASCADE (but this drops all dependent policies too!)
DROP FUNCTION is_member_of_team(uuid) CASCADE;
```

`USING (true)` means "everyone, including unauthenticated"

The anon key doesn't carry a JWT with a user ID. So `auth.uid()` returns NULL for unauthenticated requests. A policy with `USING (true)` doesn't check auth at all — it grants access to anyone who can reach the API.

---

9. Testing Your Policies

Test unauthenticated access (should return `[ ]` on protected tables)

```shell
curl
"https://your-project.supabase.co/rest/v1/your_table?select=*" \
  -H "apikey: your-anon-key" \
  -H "Authorization: Bearer your-anon-key"
```

Test authenticated access (should return only scoped data)

```shell
# Sign in to get a JWT
TOKEN=$(curl -s
"https://your-project.supabase.co/auth/v1/token?grant_type=passwo
rd" \
  -H "apikey: your-anon-key" \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"..."}' | jq -r
'.access_token')

# Query with the JWT — should only see what this user's role
allows
curl "https://your-project.supabase.co/rest/v1/members?select=*"
\
  -H "apikey: your-anon-key" \
  -H "Authorization: Bearer $TOKEN"
```

Test each role

Sign in as a user of each role (admin, manager, regular user) and verify:

- They see the data they should
- They don't see data they shouldn't
- Write operations work for their allowed scope
- Write operations fail for data outside their scope

---

10. Setup Order for New Projects

When starting a new Supabase project, do security first:

1. **Design your schema with minimal PII** — don't copy emails from `auth.users`
2. **Enable RLS on every table immediately** — `ALTER TABLE ... ENABLE ROW LEVEL SECURITY`
3. **Create helper functions** — `get_my_role()`, ownership checks
4. **Add SELECT policies first** — this is what most people forget
5. **Add INSERT/UPDATE/DELETE policies** — scope writes to the correct role
6. **Test with curl** — not your app UI, the raw API
7. **Disable schema introspection** if not needed publicly
8. **Audit regularly** — run the audit queries from section 3 after any schema change

Template: Minimal secure table

```sql
-- Create table
CREATE TABLE public.items (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES auth.users NOT NULL DEFAULT auth.uid(),
  title text NOT NULL,
  created_at timestamptz DEFAULT now()
);

-- Enable RLS
ALTER TABLE public.items ENABLE ROW LEVEL SECURITY;

-- Policies
CREATE POLICY "Users read own items"
  ON public.items FOR SELECT
  USING (auth.uid() = user_id);

CREATE POLICY "Users create own items"
  ON public.items FOR INSERT
  WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Users update own items"
  ON public.items FOR UPDATE
```

```
  USING (auth.uid() = user_id);

CREATE POLICY "Users delete own items"
  ON public.items FOR DELETE
  USING (auth.uid() = user_id);
```

---

11. Storage Buckets Have Separate RLS

Supabase Storage has its own RLS system on `storage.objects`. Database RLS does not protect your files. If you create a bucket and upload files, they need their own policies or they're either inaccessible or world-readable.

```sql
-- BAD: Public bucket with no policies — anyone can read all
files
-- (This is common when vibe-coding file upload features)

-- GOOD: Private bucket with scoped policies
CREATE POLICY "Users can read own files"
  ON storage.objects FOR SELECT
  USING (auth.uid() = owner_id);

CREATE POLICY "Users can upload own files"
  ON storage.objects FOR INSERT
  WITH CHECK (
    auth.uid() IS NOT NULL
    AND bucket_id = 'avatars'
    AND (storage.foldername(name))[1] = auth.uid()::text
  );
```

The pattern of using the user's ID as a folder name (`avatars/{user_id}/photo.png`) makes policies simple and predictable.

---

12. Realtime Subscriptions Require SELECT Policies

Supabase Realtime only sends events to users who have SELECT access to the affected rows. If you have INSERT/UPDATE policies but no SELECT policy, Realtime silently fails — users subscribe but never receive events.

```sql
-- BAD: User can insert messages but Realtime won't deliver them
CREATE POLICY "Users can insert messages"
  ON public.messages FOR INSERT
  WITH CHECK (auth.uid() = sender_id);
-- No SELECT policy → Realtime subscription returns nothing

-- GOOD: Include SELECT so Realtime works
CREATE POLICY "Users can read their conversation messages"
  ON public.messages FOR SELECT
  USING (
    conversation_id IN (
      SELECT conversation_id FROM public.participants
      WHERE user_id = auth.uid()
    )
  );
```

Also be aware that complex RLS policies (multiple joins, subqueries) slow down Realtime connections. Keep Realtime-critical policies simple.

---

13. Database Triggers Need SECURITY DEFINER

The common `handle_new_user` trigger that creates a profile when someone signs up runs as the `supabase_auth_admin` role — which doesn't have permission to write to your `public` schema. Without `SECURITY DEFINER`, the trigger fails silently and signups appear to work but no profile row is created.

```sql
-- BAD: Trigger fails silently — no profile created on signup
```

```
CREATE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
  INSERT INTO public.profiles (id, display_name)
  VALUES (new.id, new.raw_user_meta_data->>'name');
  RETURN new;
END;
$$ LANGUAGE plpgsql;

-- GOOD: SECURITY DEFINER + constrained search_path
CREATE FUNCTION public.handle_new_user()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public  -- Prevents privilege escalation via
search_path
AS $$
BEGIN
  INSERT INTO public.profiles (id, display_name)
  VALUES (new.id, new.raw_user_meta_data->>'name');
  RETURN new;
END;
$$;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

Always set `search_path` on `SECURITY DEFINER` functions. Without it, an attacker could create a malicious function in a different schema that gets called instead of the one you intended.

---

14. RPC Functions: INVOKER vs DEFINER

User-facing RPC functions (called via `supabase.rpc()`) should use `SECURITY INVOKER` (the default) so RLS is respected. Only use `SECURITY DEFINER` for internal helper functions used inside policies.

```sql
-- BAD: User-facing function bypasses all RLS
CREATE FUNCTION public.search_users(query text)
RETURNS SETOF public.profiles
LANGUAGE sql
SECURITY DEFINER  -- Runs as superuser, returns ALL matching
profiles
AS $$
  SELECT * FROM public.profiles WHERE display_name ILIKE '%' ||
query || '%';
$$;

-- GOOD: User-facing function respects RLS
CREATE FUNCTION public.search_users(query text)
RETURNS SETOF public.profiles
LANGUAGE sql
SECURITY INVOKER  -- Runs as the calling user, RLS filters
results
AS $$
  SELECT * FROM public.profiles WHERE display_name ILIKE '%' ||
query || '%';
$$;
```

Rule of thumb:

- **SECURITY INVOKER** (default): For any function users call directly. RLS applies.
- **SECURITY DEFINER**: Only for helper functions used inside RLS policies to avoid recursion. Never expose these to users via `rpc()`.

---

15. Environment Variable Leaks Across Frameworks

Every framework has different rules for which env vars are exposed to the client. Getting this wrong leaks your service role key — which bypasses all RLS.

Framework	Client-exposed prefix	Server-only
Next.js	`NEXT_PUBLIC_*`	Everything else
Nuxt	`NUXT_PUBLIC_*`	`NUXT_*` without PUBLIC
SvelteKit	`PUBLIC_*`	Everything else
Vite (React/Vue)	`VITE_*`	Everything else
Remix	Must be explicitly passed to loader	Everything in `process.env`

```shell
# .env.local — the naming matters

# These get bundled into client JavaScript (VISIBLE TO EVERYONE):
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbG...

# This stays on the server (safe):
SUPABASE_SERVICE_ROLE_KEY=eyJhbG...

# CATASTROPHIC MISTAKE — service role key shipped to every
browser:
NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY=eyJhbG...
```

If you accidentally ship the service role key, **rotate it immediately** in Supabase Dashboard > Project Settings > API.

---

16. Vibe-Coding-Specific Risks

AI coding assistants frequently introduce security vulnerabilities in Supabase apps. Here's what to watch for.

AI suggests `USING (true)` to "fix" permission errors

When an AI agent hits an RLS permission error, its instinct is to make the error go away. The fastest fix is `USING (true)` — which makes the table world-readable. Always review AI-generated policies.

```sql
-- What AI generates when it hits a permission error:
CREATE POLICY "fix_permissions" ON public.users
  FOR SELECT USING (true);  -- "Fixed!" (your database is now
public)

-- What it should generate:
CREATE POLICY "users_read_own" ON public.users
  FOR SELECT USING (auth.uid() = id);
```

AI doesn't understand the anon key security model

AI often treats the anon key like a secret and tries to "hide" it, or conversely puts the service role key in client code because "the app needs database access." Neither is correct.

AI skips RLS entirely

Many AI-generated Supabase schemas have no RLS at all. Every table in `public` schema is auto-exposed via the REST API. Without RLS, `curl` + anon key = full database dump.

Post-generation checklist

After any AI generates Supabase code, verify:

1. Is RLS enabled on every table? (`ALTER TABLE ... ENABLE ROW LEVEL SECURITY`)
2. Does every table have SELECT, INSERT, UPDATE, DELETE policies?
3. Are any policies using `USING (true)`?
4. Is the service role key only in server-side code?

5. Are there any emails, passwords, or secrets in public tables?
6. Test every table with an unauthenticated `curl` — should return `[]`

---

17. Private Schemas for Internal Tables

Tables in the `public` schema are automatically exposed via the REST API. If you have tables that should never be accessible from the client (API keys, internal config, audit logs), move them to a private schema:

```sql
-- Create a private schema (not exposed via REST API)
CREATE SCHEMA private;

-- Internal tables go here
CREATE TABLE private.api_keys (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  service_name text NOT NULL,
  key_value text NOT NULL
);

CREATE TABLE private.audit_log (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid,
  action text,
  details jsonb,
  created_at timestamptz DEFAULT now()
);

-- These tables are invisible to the REST API
-- Only accessible via database functions or server-side service
role
```

You can also disable the REST API entirely in **Project Settings > API** if your app only uses server-side access.

Summary

Principle	Why
Don't store emails in public tables	`auth.users` already has them and is never exposed via API
Enable RLS on every table	Without it, the table is fully public
Always add SELECT policies	Missing SELECT = unauthenticated full read
Use `SECURITY DEFINER` helpers	Prevents recursion, keeps policies readable
Separate sensitive columns	RLS is row-level, not column-level
Lock sensitive fields with `WITH CHECK`	Prevents role escalation and status manipulation via UPDATE
Constrain self-registration INSERTs	Users can only create records with safe default values
Enable email confirmation	Without it, "authenticated-only" means "anyone on the internet"
Test with `curl`, not your UI	Middleware and route guards are not security
Add own-record policies alongside scope policies	Users who change teams/groups keep their history

Never put service role key in `NEXT_PUBLIC_`	It bypasses all RLS
Storage buckets need their own policies	Database RLS doesn't protect files
Realtime needs SELECT policies	No SELECT = silent subscription failure
Triggers need SECURITY DEFINER + search_path	Auth triggers fail silently without it
User-facing RPCs should use SECURITY INVOKER	DEFINER bypasses RLS — only use for internal helpers
Use private schemas for internal tables	Public schema = auto-exposed via REST API
Review all AI-generated RLS policies	AI defaults to `USING (true)` to "fix" errors